reth 네트워크 모듈 흐름 정리

목차

1. 전체 아키텍처 조감

2. 설정과 부트스트랩

2.1 `NetworkConfig` 준비

2.2 `NetworkBuilder` / `NetworkManager::new`

3. 피어 디스커버리 파이프라인

3.1 서비스 스폰: `Discovery::new`

3.2 이벤트 스트림: `Discovery::poll`

4. 네트워크 상태 & 피어 관리 (`NetworkState`)

5. 스웜 루프 (`Swarm::poll_next`)

6. 세션 관리 FSM (`SessionManager`)

7. 활성 세션 처리 (`ActiveSession`)

8. 요청/응답 하위 시스템

8.1 블록 다운로드 (`StateFetcher` , `FetchClient`)

8.2 ETH 요청 처리 (`EthRequestHandler`)

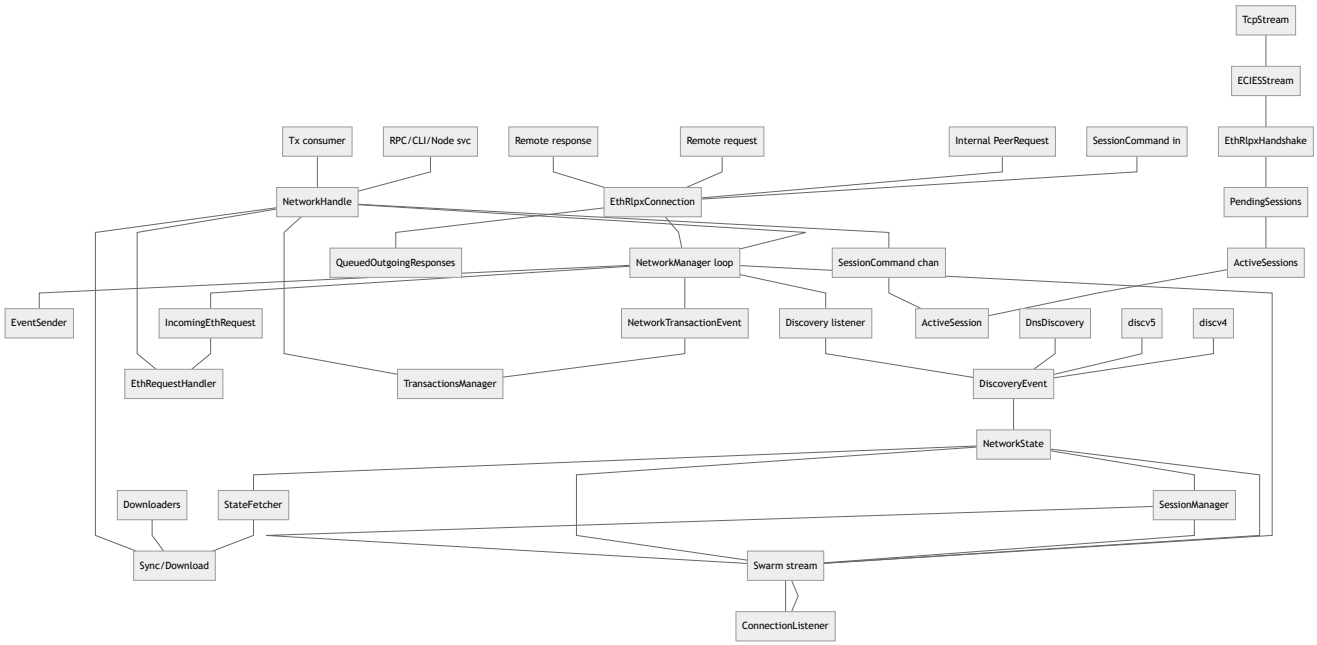
8.3 트랜잭션 전파 (`TransactionsManager`)

9. 외부 통신 인터페이스 (`NetworkHandle` , `NetworkEvent`)

10. 종료, NAT, 메트릭 및 기타 유틸

---

1. 전체 아키텍처 조감


TcpStream
ECIESStream
Tx consumer
RPC/CLI/Node svc
Remote response
Remote request
Internal PeerRequest
SessionCommand in
EthRlpxHandshake
NetworkHandle
EthRlpxConnection
PendingSessions
QueuedOutgoingResponses
NetworkManager loop
SessionCommand chan
ActiveSessions
EventSender
IncomingEthRequest
NetworkTransactionEvent
Discovery listener
ActiveSession
DnsDiscovery
discv5
discv4
EthRequestHandler
TransactionsManager
DiscoveryEvent
NetworkState
Downloaders
StateFetcher
SessionManager
Sync/Download
Swarm stream
ConnectionListener

- `NetworkManager` 가 최상위 future로 실행되며, 내부에 `Swarm` , `NetworkState` , `SessionManager` , 디스커버리 서비스, 요청/응답 채널을 보유함.
- 스웜은 “상태 → 세션 → 리스너” 순으로 내부 이벤트 우선순위를 두고 회전.
- 각 활성 세션은 독립 태스크(`ActiveSession`)로 실행되며 RLPx/ETH 메시지를 처리.
- 외부 모듈(동기화, 트랜잭션 풀, RPC)은 `NetworkHandle` 과 이벤트 스트림으로 네트워크와 상호작용.

2. 설정과 부트스트랩

2.1 `NetworkConfig` 준비

- 키, 리스너 주소, 부트노드, 세션 제한, 추가 프로토콜, NAT 정책 등이 이 구조체에 모임.
- `NetworkConfig::builder` 체이닝으로 작성한 뒤 `build` 호출 시 클라이언트 (`BlockReader`)를 입력.

2.2 `NetworkBuilder` / `NetworkManager::new`

- `NetworkBuilder` 는 `NetworkManager` , `TransactionsManager` , `EthRequestHandler` 를 한꺼번에 엮어주는 도우미.
- `NetworkManager::new` 가 구조체 인스턴스를 만드는 핵심 함수.

```
// crates/net/network/src/manager.rs:229

pub  async  fn new<C: BlockNumReader + 'static>(

config: NetworkConfig<C, N>,

) -> Result<Self, NetworkError> {

let NetworkConfig { client, secret_key, discovery_v4_addr, mut discovery_v4_conf



let peers_manager = PeersManager::new(peers_config);
```

```
let peers_handle = peers_manager.handle();


let incoming = ConnectionListener::bind(listener_addr).await?;

…

let discovery = Discovery::new(

listener_addr,

discovery_v4_addr,

secret_key,

discovery_v4_config,

discovery_v5_config,

dns_discovery_config,

).await?;


let sessions = SessionManager::new(

secret_key, sessions_config, executor, status, hello_message,

fork_filter, extra_protocols, handshake,

);


let state = NetworkState::new(

BlockNumReader::new(client),

discovery,

peers_manager,

Arc::clone(&num_active_peers),
```

```
);

let swarm = Swarm::new(incoming, sessions, state);
…
Ok(Self { swarm, handle, … })
}
```

- 구성 직후 `NetworkBuilder::split_with_handle` 로 `(NetworkHandle, NetworkManager, TransactionsManager, EthRequestHandler)` 를 받아 태스크로 실행.

3. 피어 디스커버리 파이프라인

3.1 서비스 스폰: `Discovery::new`

- discv4, discv5, DNS(EIP-1459) 서비스 각각을 비동기 태스크로 띄움.
- Bootnode 설정과 ForkId 세팅을 초기화 시점에 적용.

```
// crates/net/network/src/discovery.rs:74
pub  async  fn new(
tcp_addr: SocketAddr,
discovery_v4_addr: SocketAddr,
sk: SecretKey,
discv4_config: Option<Discv4Config>,
discv5_config: Option<reth_discv5::Config>,
dns_discovery_config: Option<DnsDiscoveryConfig>,
```

```
) -> Result<Self, NetworkError> {
let local_enr =
NodeRecord::from_secret_key(discovery_v4_addr, &sk).with_tcp_port(tcp_addr.port(


let (discv4, discv4_updates, _discv4_service) = Discv4::bind(...).await?;
let (discv5, discv5_updates) = Discv5::start(&sk, config).await?;
let (dns_handle, dns_updates, dns_task) = DnsDiscoveryService::new_pair(...);
…
Ok(Self { discv4, discv5, dns_discovery_updates: dns_updates, … })
}
```

3.2 이벤트 스트림: `Discovery::poll`

- 각 서비스의 업데이트 스트림을 폴링하여 `DiscoveryEvent` 큐에 넣고, 리스너들에게 브로드캐스트.

```
// crates/net/network/src/discovery.rs:251
pub(crate) fn poll(&mut  self, cx: &mut Context<'_>) -> Poll<DiscoveryEvent> {
if  let Some(event) = self.queued_events.pop_front() {
self.notify_listeners(&event);
return Poll::Ready(event)
}


while  let Some(Poll::Ready(Some(update))) =
```

```
self.discv4_updates.as_mut().map(|u| u.poll_next_unpin(cx))
{
self.on_discv4_update(update);
}
…
if  self.queued_events.is_empty() {
return Poll::Pending
}
}
```

- 결과물은 `DiscoveryEvent::DiscoveredNode`, `DiscoveryEvent::EnrForkId` 등으로 `NetworkState` 로 전달됨.

4. 네트워크 상태 & 피어 관리 (`NetworkState`)

- `PeersManager` 와 `StateFetcher`, `Discovery` 를 캡슐화해 네트워크 전체 상태를 관리.
- `poll` 실행 순서:

1. 버퍼된 `StateAction` 전송
2. `Discovery::poll` 로 신규 노드 감지
3. `StateFetcher::poll` 로 블록 다운로드 요청 처리
4. 활성 세션의 응답 채널(`PeerResponse`) 확인
5. `PeersManager::poll` 로 연결 정책 적용.

```
// crates/net/network/src/state.rs:428
pub(crate) fn poll(&mut  self, cx: &mut Context<'_>) -> Poll<StateAction<N>> {
```

```
if  let Some(message) = self.queued_messages.pop_front() {
return Poll::Ready(message)
}


while  let Poll::Ready(discovery) = self.discovery.poll(cx) {
self.on_discovery_event(discovery);
}
while  let Poll::Ready(action) = self.state_fetcher.poll(cx) {
match action {
FetchAction::BlockRequest { peer_id, request } => self.handle_block_request(peer
}
}
…
while  let Poll::Ready(action) = self.peers_manager.poll(cx) {
self.on_peer_action(action);
}
…
Poll::Pending
}
```

- `StateAction::Connect`, `StateAction::NewBlock` 와 같은 출력은 `Swarm` 에 의해 후속 처리됨.

5. 스웜 루프 (`Swarm::poll_next`)

- `Swarm` 은 `NetworkState`, `SessionManager`, `ConnectionListener` 세 가지를 우선순위로 폴링.
- `StateAction` 을 받아 다이얼 요청, 메시지 전송 등을 실행하고, 그 결과를 `SwarmEvent` 로 `NetworkManager` 에 알림.

```
// crates/net/network/src/swarm.rs:292

fn poll_next(self: Pin<&mut  Self>, cx: &mut Context<'_>) -> Poll<Option<SwarmEv

let this = self.get_mut();

while  let Poll::Ready(action) = this.state.poll(cx) {

if  let Some(event) = this.on_state_action(action) {

return Poll::Ready(Some(event))

}

}

match this.sessions.poll(cx) {

Poll::Ready(event) => {

if  let Some(event) = this.on_session_event(event) {

return Poll::Ready(Some(event))

}

}

_ => {}

}

match Pin::new(&mut this.incoming).poll(cx) {

Poll::Ready(event) => {
```

```
if let Some(event) = this.on_connection(event) {
return Poll::Ready(Some(event))
}
}
_ => {}
}
Poll::Pending
}
```

6. 세션 관리 FSM (`SessionManager`)

- Pending/Active 세션을 맵으로 분리 관리, 세션별 커맨드 채널을 생성.
- Outbound 다이얼 요청은 `dial_outbound` 에서 `pending_session_with_timeout` 태스크를 스폰.
- `poll` 함수로 활성/대기 세션에서 올라오는 메시지를 `SessionEvent` 로 변환.

```
// crates/net/network/src/session/mod.rs:307
pub fn dial_outbound(&mut self, remote_addr: SocketAddr, remote_peer_id: PeerI
if self.counter.ensure_pending_outbound().is_ok() {
let session_id = self.next_id();
let (disconnect_tx, disconnect_rx) = oneshot::channel();
let pending_events = self.pending_sessions_tx.clone();
let secret_key = self.secret_key;
let hello_message = self.hello_message.clone();
```

```
let fork_filter = self.fork_filter.clone();

let status = self.status;

let extra_handlers = self.extra_protocols.on_outgoing(remote_addr, remote_peer_i

self.spawn(pending_session_with_timeout(

self.pending_session_timeout,

session_id,

remote_addr,

Direction::Outgoing(remote_peer_id),

pending_events.clone(),

start_pending_outbound_session(

self.handshake.clone(),

disconnect_rx,

pending_events,

session_id,

remote_addr,

remote_peer_id,

secret_key,

hello_message,

status,

fork_filter,

extra_handlers,

),

));
```

```
…
self.counter.inc_pending_outbound();
}
}


// crates/net/network/src/session/mod.rs:439
pub(crate) fn poll(&mut  self, cx: &mut Context<'_>) -> Poll<SessionEvent<N>> {
match  self.active_session_rx.poll_next_unpin(cx) {
Poll::Ready(Some(event)) => { … return Poll::Ready(SessionEvent::ValidMessage {
_ => {}
}
let event = match  self.pending_session_rx.poll_next_unpin(cx) {
Poll::Ready(Some(event)) => event,
Poll::Pending => return Poll::Pending,
_ => unreachable!(),
};
match event {
PendingSessionEvent::Established { session_id, peer_id, capabilities, conn, … }
…
Poll::Ready(SessionEvent::SessionEstablished { peer_id, capabilities, … })
}
PendingSessionEvent::Disconnected { … } => { … }
```

```
PendingSessionEvent::OutgoingConnectionError { … } => { … }

}

}
```

- 핸드셰이크 단계에서는 `TcpStream → ECIESStream → EthRlpxHandshake` 순으로 진행.

7. 활성 세션 처리 (`ActiveSession`)

- 활성 세션 태스크는 네 가지 이벤트 소스를 동시에 감시:

1. `SessionCommand` (SessionManager 지시)
2. 내부 요청 (`PeerRequestSender`)
3. 원격에서 들어온 요청(ETH 메시지)
4. 원격 응답

- 수신 메시지를 `PeerMessage` 로 감싸 `SessionManager` 에 전달하고, 요청 큐를 관리해 백프레셔를 구현.

```
// crates/net/network/src/session/active.rs:157

fn on_incoming_message(&mut  self, msg: EthMessage<N>) -> OnIncomingMessageOutco

macro_rules! on_request {

($req:ident, $resp_item:ident, $req_item:ident) => {{

let RequestPair { request_id, message: request } = $req;

let (tx, response) = oneshot::channel();

let received = ReceivedRequest { request_id, rx: PeerResponse::$resp_item { resp

self.received_requests_from_remote.push(received);

self.try_emit_request(PeerMessage::EthRequest(PeerRequest::$req_item { request,
```

```
}}

}

macro_rules! on_response { … }

match msg {

EthMessage::NewBlock(block) => { … }

EthMessage::GetBlockBodies(req) => on_request!(req, BlockBodies, GetBlockBodies)

EthMessage::BlockBodies(resp) => on_response!(resp, GetBlockBodies),

…

}

}
```

- `QueuedOutgoingResponses` 로 대용량 응답(`Bodies` , `Receipts` 등) 수를 제한 (`MAX_QUEUED_OUTGOING_RESPONSES = 4`).

8. 요청/응답 하위 시스템

8.1 블록 다운로드 (`StateFetcher` , `FetchClient`)

- `NetworkState` 는 `StateFetcher::new` 를 통해 세션별 `PeerRequestSender` 를 유지.
- 외부 동기화 모듈은 `FetchClient` 핸들을 받아 `GetBlockHeaders` , `GetBlockBodies` 요청을 전송.
- 응답은 `PeerResponse` 채널을 통해 다시 `StateFetcher` 로 돌아와 후속 요청/평판 조치 결정.

8.2 ETH 요청 처리 (`EthRequestHandler`)

- `NetworkBuilder::request_handler` 가 `NetworkManager` 와 채널을 연결.

- `SessionManager` 에서 원격 요청을 받으면 `NetworkManager::on_peer_message` → `on_eth_request` → `IncomingEthRequest` 채널로 내려감.
- 핸들러는 스토리지/클라이언트(`BlockReader` , `BlockWriter`)를 사용해 응답 생성 후 다시 `NetworkHandleMessage::EthRequest` 로 회신.

8.3 트랜잭션 전파 (`TransactionsManager`)

- `NetworkBuilder::transactions` 에서 네트워크 핸들을 주입하고 브로드캐스트 정책 (`NetworkPolicies`)을 설정.
- 원격으로부터 들어온 트랜잭션 이벤트는 `NetworkManager::on_peer_message -> NetworkTransactionEvent` 로 전달.
- 트랜잭션 풀에서 새 해시/트랜잭션 전파 시 `NetworkHandleMessage::SendPooledTransactionHashes` 등의 메시지가 올라와 세션에 전달.

9. 외부 통신 인터페이스 (`NetworkHandle` , `NetworkEvent`)

- `NetworkHandle` 은 다수의 `NetworkHandleMessage` 를 통해 네트워크에 요청:
- `ConnectPeer` , `DisconnectPeer` , `AddTrustedPeerId` , `EthRequest` , `AnnounceBlock` , `FetchClient` 요청 등.
- `NetworkManager::on_handle_message` 가 메시지를 받아 적절한 내부 모듈에 위임.

```
// crates/net/network/src/manager.rs:647

fn on_handle_message(&mut  self, msg: NetworkHandleMessage<N>) {

match msg {

NetworkHandleMessage::DiscoveryListener(tx) => {

self.swarm.state_mut().discovery_mut().add_listener(tx);

}
```

```
NetworkHandleMessage::ConnectPeer(peer_id, kind, addr) => {

self.swarm.state_mut().add_and_connect(peer_id, kind, addr);

}

NetworkHandleMessage::EthRequest { peer_id, request } => {

self.swarm.sessions_mut().send_message(&peer_id, PeerMessage::EthRequest(request

}

…

}

}
```

- 이벤트 출력은 `NetworkEvent` (예: `ActivePeerSession`, `PeerEvent::PeerAdded`, `PeerEvent::PeerRemoved`) 로 외부에 스트림 형태로 전달됨.

10. 종료, NAT, 메트릭 및 기타 유틸

- `NetworkManager::perform_network_shutdown` 이 Discovery, Session, Swarm을 정리하고 모든 세션을 `DisconnectReason::ClientQuitting` 으로 종료.
- NAT 지원: `crates/net/nat` 의 `NatResolver` 가 ENR/노드 정보 갱신에 사용.
- 메트릭: 각 모듈(`SessionManagerMetrics`, `NetworkMetrics`)이 `prometheus` / `metrics` 계측을 제공.
- Ban/평판: `crates/net/banlist`, `PeersManager::apply_reputation_change` 로 관리.

참고: 실행 흐름 요약

1. 설정 단계에서 `NetworkConfig` → `NetworkManager` → `Swarm` → `Discovery`, `SessionManager`, `NetworkState` 가 구성된다.

2. 런타임은 `tokio::spawn(network_manager)` 로 구동된다.

3. `NetworkManager::poll` 는 (a) 핸들 명령 처리, (b) `Swarm` 이벤트 처리, © 블록 import 작업을 순서대로 수행한다.

4. `Swarm` 은 `NetworkState` → `SessionManager` → `ConnectionListener` 순으로 이벤트를 소화하고 결과를 `SwarmEvent` 로 반환한다.

5. `SessionManager` 는 Pending/Active 세션을 FSM으로 관리하며, ECIES/RLPx 핸드셰이크를 거쳐 `ActiveSession` 태스크를 스폰한다.

6. `ActiveSession` 은 메시지를 파싱하여 `PeerMessage` 를 상위로 전달하고, 내부 요청/응답 큐를 통해 동기화·트랜잭션 하위 시스템과 협력한다.

7. 외부 모듈은 `NetworkHandle` 과 `NetworkEvent` 스트림을 통해 네트워크에 명령하거나 상태를 관찰한다.

8. 종료 시 모든 세션과 discovery 서비스가 정리되어 안전하게 내려간다.

이 문서를 참고하면 `crates/net` 내 주요 모듈이 어떤 책임을 가지고 어떻게 상호작용하는지 전체적으로 파악할 수 있습니다.